Salma Elmaghawry
Software Engineer & Content Creator

// flutter · study guide

Local Notifications

One app, built step by step. By the end, you have a working Study Reminder on your phone.

---

01 What & Why Notifications

02 Setup & First Notification

03 Scheduled Notifications

04 Handling Notification Taps

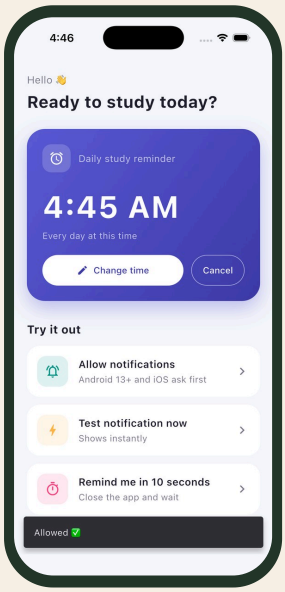
Every code block is complete. Copy it as is.

// the app

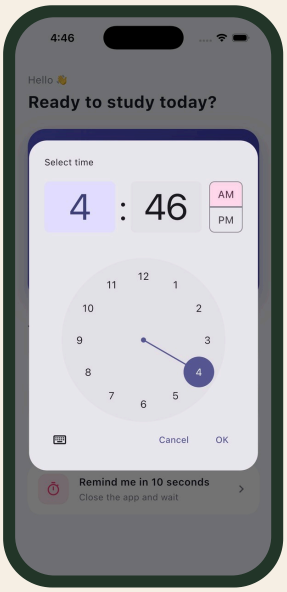
The App

You'll build an app that reminds you to study **every day** at a time you choose. Tapping the reminder opens a motivation screen.

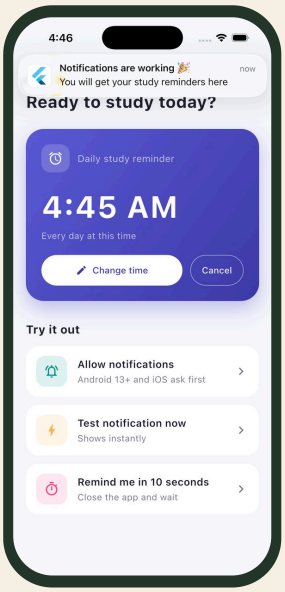
SCREEN	WHAT'S ON IT
Home	Greeting, a purple **reminder card** (time + Pick / Change / Cancel), and a **"Try it out"** list
Motivation	Opens when you tap a notification: a trophy, a random motivational message, the **payload** chip, and a "Let's study 📖 " button



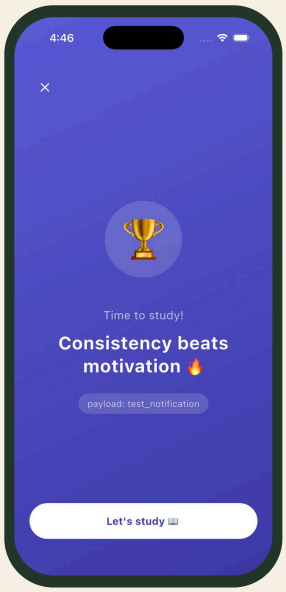
// 01 · allow



// 02 · pick a time



// 03 · reminder arrives



// 04 · tap it

Screenshots taken on iPhone.

Final project structure

```
// STRUCTURE

lib/
├── main.dart                    # init + tap handling + MaterialApp
├── core/
│   ├── app_theme.dart           # colors + theme                (starter)
│   └── reminder_ids.dart        # notification ids              (Part 2)
├── routing/
│   ├── routes.dart              # route names                   (Part 4)
│   └── app_router.dart          # generateRoute()               (Part 4)
├── services/
│   └── notification_service.dart  # ⭐ everything about notifications (Parts 2-4)
├── screens/
│   ├── home_screen.dart         # buttons → service calls       (starter + Parts 2-3)
│   └── motivation_screen.dart   # opened from a notification    (Part 4)
└── widgets/
    ├── reminder_card.dart       # purple card                   (starter)
    └── action_tile.dart         # white tile with icon          (starter)
```

// overview

What You'll Learn

#	TOPIC	WHAT YOU'LL HAVE AT THE END
1	**What & Why Notifications**	The concepts behind notifications
2	**Setup & First Notification**	"Test notification now" works 🎉
3	**Scheduled Notifications**	A daily reminder that works even when the app is closed
4	**Handling Notification Taps**	Tapping a notification opens the Motivation screen
+	**Running on iPhone**	The same app working on iOS

// start here

Starter Files

These files are **UI only**, so you can focus on notifications. Create a new Flutter project named `local_notifications_app` and add these 4 files.

lib/core/app_theme.dart

```dart
// DART
import 'package:flutter/material.dart';

class AppColors {
  static const primary = Color(0xFF5B5BD6);
  static const primaryDark = Color(0xFF3E3BA8);
  static const accent = Color(0xFFFFB547);
  static const background = Color(0xFFF5F6FB);
  static const textDark = Color(0xFF1E1E2D);
  static const textMuted = Color(0xFF7A7A8C);

  static const heroGradient = LinearGradient(
    colors: [primary, primaryDark],
    begin: Alignment.topLeft,
    end: Alignment.bottomRight,
  );
}

class AppTheme {
  static ThemeData get light => ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(seedColor: AppColors.primary),
    scaffoldBackgroundColor: AppColors.background,
    snackBarTheme: const SnackBarThemeData(behavior: SnackBarBehavior.floating),
  );
}
```

lib/widgets/action_tile.dart

```dart
// DART
import 'package:flutter/material.dart';
import 'package:local_notifications_app/core/app_theme.dart';

/// A white rounded row with a colored icon, used for the test actions.
class ActionTile extends StatelessWidget {
  const ActionTile({
    super.key,
    required this.icon,
    required this.color,
    required this.title,
    required this.subtitle,
    required this.onTap,
  });

  final IconData icon;
```

```
  final Color color;
  final String title;
  final String subtitle;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 12),
      child: Material(
        color: Colors.white,
        borderRadius: BorderRadius.circular(20),
        child: InkWell(
          borderRadius: BorderRadius.circular(20),
          onTap: onTap,
          child: Padding(
            padding: const EdgeInsets.all(16),
            child: Row(
              children: [
                Container(
                  padding: const EdgeInsets.all(12),
                  decoration: BoxDecoration(
                    color: color.withValues(alpha: 0.12),
                    borderRadius: BorderRadius.circular(14),
                  ),
                  child: Icon(icon, color: color),
                ),
                const SizedBox(width: 16),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        title,
                        style: const TextStyle(
                          fontSize: 16,
                          fontWeight: FontWeight.w600,
                          color: AppColors.textDark,
                        ),
                      ),
                      const SizedBox(height: 2),
                      Text(
                        subtitle,
                        style: const TextStyle(color: AppColors.textMuted),
                      ),
                    ],
                  ),
                ),
                const Icon(Icons.chevron_right, color: AppColors.textMuted),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/widgets/reminder_card.dart

```dart
// DART

import 'package:flutter/material.dart';
import 'package:local_notifications_app/core/app_theme.dart';

/// The big purple card on top of Home that shows the daily reminder.
class ReminderCard extends StatelessWidget {
  const ReminderCard({
    super.key,
    required this.reminderTime,
    required this.onPickTime,
    required this.onCancel,
  });

  final TimeOfDay? reminderTime;
  final VoidCallback onPickTime;
  final VoidCallback onCancel;

  @override
  Widget build(BuildContext context) {
    final hasReminder = reminderTime != null;

    return Container(
      padding: const EdgeInsets.all(24),
      decoration: BoxDecoration(
        gradient: AppColors.heroGradient,
        borderRadius: BorderRadius.circular(28),
        boxShadow: [
          BoxShadow(
            color: AppColors.primary.withValues(alpha: 0.35),
            blurRadius: 24,
            offset: const Offset(0, 12),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Container(
                padding: const EdgeInsets.all(10),
                decoration: BoxDecoration(
                  color: Colors.white.withValues(alpha: 0.18),
                  borderRadius: BorderRadius.circular(14),
                ),
                child: const Icon(Icons.alarm, color: Colors.white),
              ),
              const SizedBox(width: 12),
              const Text(
                'Daily study reminder',
                style: TextStyle(color: Colors.white70, fontSize: 15),
              ),
            ],
          ),
          const SizedBox(height: 20),
          Text(
            hasReminder ? reminderTime!.format(context) : '--:--',
```

```
            style: const TextStyle(
              color: Colors.white,
              fontSize: 48,
              fontWeight: FontWeight.bold,
              letterSpacing: 1,
            ),
          ),
          Text(
            hasReminder ? 'Every day at this time' : 'No reminder set yet',
            style: const TextStyle(color: Colors.white70),
          ),
          const SizedBox(height: 20),
          Row(
            children: [
              Expanded(
                child: FilledButton.icon(
                  onPressed: onPickTime,
                  icon: Icon(hasReminder ? Icons.edit : Icons.add_alarm),
                  label: Text(hasReminder ? 'Change time' : 'Pick study time'),
                  style: FilledButton.styleFrom(
                    backgroundColor: Colors.white,
                    foregroundColor: AppColors.primaryDark,
                    padding: const EdgeInsets.symmetric(vertical: 14),
                  ),
                ),
              ),
              if (hasReminder) ...[
                const SizedBox(width: 12),
                OutlinedButton(
                  onPressed: onCancel,
                  style: OutlinedButton.styleFrom(
                    foregroundColor: Colors.white,
                    side: const BorderSide(color: Colors.white54),
                    padding: const EdgeInsets.symmetric(
                      vertical: 14,
                      horizontal: 18,
                    ),
                  ),
                  child: const Text('Cancel'),
                ),
              ],
            ],
          ),
        ],
      ),
    );
  }
}
```

`lib/screens/home_screen.dart` (starter version)

The UI is complete. Each method is empty with a `// TODO`, and you'll fill them in during Parts 2 and 3.

> This file won't compile until you create `reminder_ids.dart` and `notification_service.dart` in Part 2. That's expected.

```dart
// DART

import 'package:flutter/material.dart';
import 'package:local_notifications_app/core/app_theme.dart';
import 'package:local_notifications_app/core/reminder_ids.dart';
import 'package:local_notifications_app/services/notification_service.dart';
import 'package:local_notifications_app/widgets/action_tile.dart';
import 'package:local_notifications_app/widgets/reminder_card.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  TimeOfDay? _reminderTime;

  void _showMessage(String text) {
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(SnackBar(content: Text(text)));
  }

  // <<---------------------- Part 2: Permission ---------->>
  Future<void> _allowNotifications() async {
    // TODO
  }

  // <<---------------------- Part 2: First notification ---------->>
  Future<void> _testNotification() async {
    // TODO
  }

  // <<---------------------- Part 3: Quick scheduled test ---------->>
  Future<void> _remindIn10Seconds() async {
    // TODO
  }

  // <<---------------------- Part 3: Daily reminder ---------->>
  Future<void> _pickStudyTime() async {
    // TODO
  }

  // <<---------------------- Part 3: Cancel only the daily reminder ---------->>
  Future<void> _cancelReminder() async {
    // TODO
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.fromLTRB(20, 24, 20, 32),
          children: [
            const Text(
              'Hello 👋',
              style: TextStyle(fontSize: 16, color: AppColors.textMuted),
```

```
            ),
            const SizedBox(height: 4),
            const Text(
              'Ready to study today?',
              style: TextStyle(
                fontSize: 26,
                fontWeight: FontWeight.bold,
                color: AppColors.textDark,
              ),
            ),
            const SizedBox(height: 24),
            ReminderCard(
              reminderTime: _reminderTime,
              onPickTime: _pickStudyTime,
              onCancel: _cancelReminder,
            ),
            const SizedBox(height: 32),
            const Text(
              'Try it out',
              style: TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: AppColors.textDark,
              ),
            ),
            const SizedBox(height: 12),
            ActionTile(
              icon: Icons.notifications_active_outlined,
              color: Colors.teal,
              title: 'Allow notifications',
              subtitle: 'Android 13+ and iOS ask first',
              onTap: _allowNotifications,
            ),
            ActionTile(
              icon: Icons.bolt,
              color: AppColors.accent,
              title: 'Test notification now',
              subtitle: 'Shows instantly',
              onTap: _testNotification,
            ),
            ActionTile(
              icon: Icons.timer_outlined,
              color: Colors.pinkAccent,
              title: 'Remind me in 10 seconds',
              subtitle: 'Close the app and wait',
              onTap: _remindIn10Seconds,
            ),
          ],
        ),
      ),
    );
  }
}
```

// part 01

What & Why Notifications

What is a notification?

A message shown to the user **outside** the app (status bar, lock screen), even when the app is closed.

Why do we use them?

- **Reminders**: study time, medicine, alarms ⬅️ *our app*
- **Re-engagement**: Duolingo's "You haven't practiced today 🦉 "
- **Event updates**: "Your order has arrived", "New message"

Ways to make a notification

WAY	WHO CREATES IT	EXAMPLE
Local	The device itself, no internet or server	Alarm, study reminder
Push (FCM)	A server	A WhatsApp message
Background + Local	Background code that then shows a local notification	A weather app checking every hour

Does our app need a server? No. The phone already knows the time, so a **local** notification is enough.

Types of local notifications, mapped to our app

TYPE	IN OUR APP
Instant	⚡ "Test notification now"
Scheduled (once)	⏱️ "Remind me in 10 seconds"
Repeating (daily)	⏰ The reminder card → "Pick study time"

Anatomy of a notification

- `id`: the notification's identity. A new notification with the same id replaces the old one. You also use the id to cancel.
- `title` / `body`: the text.
- **Channel**: the category. On Android the user can turn each channel off separately. Ours is **"Study Reminders"**.

- **Payload**: hidden data attached to the notification. The user never sees it, but you read it when they tap. Our app shows it on the Motivation screen.

Permissions

On Android 13+ and iOS, the user must **allow** notifications first. If they deny, nothing shows and **no error** appears.

> 💡 **Think of it this way:** a local notification is like an alarm you set on your own phone. A push notification is like someone calling you from outside.

// part 02

Setup & First Notification

Goal: show your first notification.

2.1 Packages

```bash
// BASH
flutter pub add flutter_local_notifications timezone flutter_timezone
```

- `flutter_local_notifications` shows the notifications.
- `timezone` + `flutter_timezone`: scheduled notifications have to use the **phone's** timezone.

2.2 `android/app/build.gradle.kts`

Add `isCoreLibraryDesugaringEnabled = true` inside `compileOptions`, and add the `dependencies` block at the end of the file:

```kotlin
// KOTLIN
android {
    // ... keep everything that's already here ...

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
        isCoreLibraryDesugaringEnabled = true
    }

    // ... keep everything that's already here ...
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")
}
```

> The package needs **desugaring** to support older Android versions. Without it, the build fails.

2.3 `android/app/src/main/AndroidManifest.xml`

This is the full file. The new parts are the **3 permissions** at the top and the **2 receivers** at the bottom of `<application>`. The app label is also changed to "Study Reminder".

```xml
// XML
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Android 13+: the user must allow notifications -->
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

```
    <!-- Reschedule notifications after the phone restarts -->
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <!-- Android 12+: fire scheduled notifications at the exact time -->
    <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM"/>

    <application
        android:label="Study Reminder"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTop"
            android:taskAffinity=""
            android:theme="@style/LaunchTheme"

android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layou
tDirection|fontScale|screenLayout|density|uiMode"
            android:hardwareAccelerated="true"
            android:windowSoftInputMode="adjustResize">
            <meta-data
              android:name="io.flutter.embedding.android.NormalTheme"
              android:resource="@style/NormalTheme"
              />
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <!-- Don't delete the meta-data below.
             This is used by the Flutter tool to generate GeneratedPluginRegistrant.java -->
        <meta-data
            android:name="flutterEmbedding"
            android:value="2" />

        <!-- flutter_local_notifications: these receivers show scheduled notifications -->
        <receiver android:exported="false"
android:name="com.dexterous.flutterlocalnotifications.ScheduledNotificationReceiver" />
        <receiver android:exported="false"
android:name="com.dexterous.flutterlocalnotifications.ScheduledNotificationBootReceiver">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED"/>
                <action android:name="android.intent.action.MY_PACKAGE_REPLACED"/>
                <action android:name="android.intent.action.QUICKBOOT_POWERON" />
                <action android:name="com.htc.intent.action.QUICKBOOT_POWERON"/>
            </intent-filter>
        </receiver>
    </application>
    <queries>
        <intent>
            <action android:name="android.intent.action.PROCESS_TEXT"/>
            <data android:mimeType="text/plain"/>
        </intent>
    </queries>
</manifest>
```

> ⚠️ **Most common mistake:** if a receiver is missing, scheduled notifications **silently** never show, and you get no error.

2.4 `lib/core/reminder_ids.dart`

```dart
// DART

/// Every notification has an id. Same id = replaces the old one,
/// and we use it to cancel ONE reminder without touching the others.
class ReminderIds {
  static const test = 0;
  static const quick = 1;
  static const daily = 2;
}
```

> **Why constants?** In Part 3, "Cancel" must remove **only** the daily reminder. Hardcoded numbers scattered around the code get mixed up.

2.5 `lib/services/notification_service.dart` (Part 2 version)

```dart
// DART

import 'package:flutter/foundation.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:flutter_timezone/flutter_timezone.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;

class NotificationService {
  NotificationService();
  static final FlutterLocalNotificationsPlugin _plugin =
      FlutterLocalNotificationsPlugin();
  static void Function(String? payload)?
  onNotificationClick; // Initialize the notification service
  //  Details for Android notifications
  static const AndroidNotificationDetails
  _androidDetails = AndroidNotificationDetails(
    'study_reminders',
    'Study Reminders', //name shown in the notification settings on the device
    channelDescription:
        'Daily reminders to study', //description shown in the notification settings on the device
    importance: Importance.max,
    priority: Priority.high,
  );

  static const NotificationDetails _details = NotificationDetails(
    android: _androidDetails,
    iOS: DarwinNotificationDetails(
      presentAlert: true,
      presentBanner: true,
      presentList: true,
      presentBadge: true,
```

```
      presentSound: true,
    ),
  );

  //<<  ----              init ----- >>
  static Future<void> init() async {
    tz.initializeTimeZones();
    final timeZone = await FlutterTimezone.getLocalTimezone();
    tz.setLocalLocation(tz.getLocation(timeZone.identifier));

    const settings = InitializationSettings(
      android: AndroidInitializationSettings('@mipmap/ic_launcher'),
      iOS: DarwinInitializationSettings(
        requestAlertPermission: false,
        requestBadgePermission: false,
        requestSoundPermission: false,
      ),
    );

    await _plugin.initialize(
      settings: settings,
      onDidReceiveNotificationResponse: (details) {
        onNotificationClick?.call(details.payload);
      },
    );
  }
  // <<---------------------- Request Permisssion ---------->>

  static Future<bool> requestPermission() async {
    if (defaultTargetPlatform == TargetPlatform.android) {
      final android = _plugin
          .resolvePlatformSpecificImplementation<
            AndroidFlutterLocalNotificationsPlugin
          >();
      return await android?.requestNotificationsPermission() ?? false;
    }
    final ios = _plugin
        .resolvePlatformSpecificImplementation<
          IOSFlutterLocalNotificationsPlugin
        >();

    return await ios?.requestPermissions(
          alert: true,
          badge: true,
          sound: true,
        ) ??
        false;
  }

  // <<---------------------- Show Notification now ---------->>

  static Future<void> showNotification({
    required int id,
    required String title,
    required String body,
    String? payload,
  }) async {
    await _plugin.show(
      id: id,
      title: title,
```

```
      body: body,
      notificationDetails: _details,
      payload: payload,
    );
  }
}
```

How it works, top to bottom:

1. `_plugin`: the single `FlutterLocalNotificationsPlugin` instance the whole app uses.
2. `onNotificationClick`: a callback that runs when the user taps a notification. You'll connect it in Part 4.
3. `_androidDetails`: the **channel** (`'study_reminders'`, shown to the user as "Study Reminders"). `Importance.max` makes the notification pop up on top of the screen.

   > Once the channel is created on the first run, its importance and sound **can't be changed from code**. Uninstall the app to reset it.

4. `init()`: sets the timezone, then initializes the plugin with the Android and iOS settings.
5. `requestPermission()`: on Android 13+ this shows the "Allow notifications?" dialog. On iOS it asks for alert, badge, and sound.
6. `showNotification()`: shows a notification **right now**.

> 📌 This is package **v22**, where all parameters are **named** (`show(id: 1, ...)`). Code from older tutorials (`show(1, ...)`) won't compile for that reason.

2.6 `lib/main.dart` (Part 2 version)

Delete the counter app and replace the whole file:

```dart
// DART
import 'package:flutter/material.dart';
import 'package:local_notifications_app/core/app_theme.dart';
import 'package:local_notifications_app/screens/home_screen.dart';
import 'package:local_notifications_app/services/notification_service.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await NotificationService.init();

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Study Reminder',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      home: const HomeScreen(),
    );
  }
}
```

> `WidgetsFlutterBinding.ensureInitialized()` is required whenever you call plugin code before `runApp`.

2.7 `home_screen.dart`: fill in the Part 2 methods

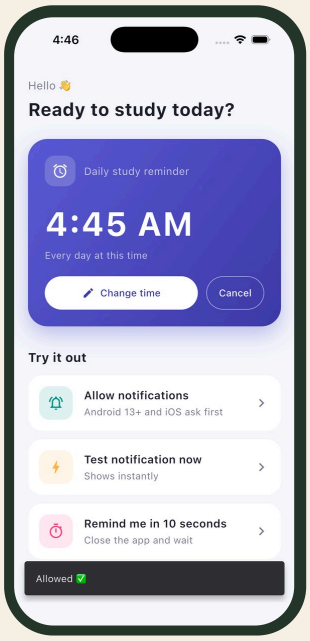
Replace the two empty methods with these:

```dart
// DART
  // <<---------------------- Part 2: Permission ---------->>
  Future<void> _allowNotifications() async {
    final granted = await NotificationService.requestPermission();
    if (mounted) _showMessage(granted ? 'Allowed ✅' : 'Denied ❌');
  }
```

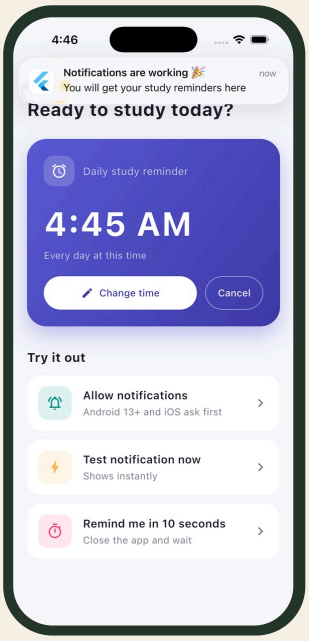
```dart
// DART

  // <<---------------------- Part 2: First notification ---------->>
  Future<void> _testNotification() {
    return NotificationService.showNotification(
      id: ReminderIds.test,
      title: 'Notifications are working 🎉',
      body: 'You will get your study reminders here',
      payload: 'test_notification',
    );
  }
```

✓ **Checkpoint:** run the app → tap **Allow notifications** → allow → tap **Test notification now** → 🎉 your first notification.



// after allow



// your first notification

Try this:

- Tap "Test notification now" **twice**. Only one notification shows, because both have the same `id`.
- Deny the permission (or turn it off in settings) and try again. Nothing shows and no error appears.

// part 03

Scheduled Notifications

Goal: a daily reminder that works even when the app is closed.

3.1 `notification_service.dart`: add the scheduling methods

Add these inside the class, after `showNotification()`:

```dart
// DART

  // <<---------------------- Schedule Notification ---------->>
  static Future<AndroidScheduleMode> _scheduleMode() async {
    if (defaultTargetPlatform ≠ TargetPlatform.android) {
      return AndroidScheduleMode.exactAllowWhileIdle;
    }
    final android = _plugin
        .resolvePlatformSpecificImplementation<
          AndroidFlutterLocalNotificationsPlugin
        >();
    var canExact = await android?.canScheduleExactNotifications() ?? false;
    if (!canExact) {
      await android?.requestExactAlarmsPermission();
      canExact = await android?.canScheduleExactNotifications() ?? false;
    }
    return canExact
        ? AndroidScheduleMode.exactAllowWhileIdle
        : AndroidScheduleMode.inexactAllowWhileIdle;
  }

  static Future<void> scheduleAfter({
    required int id,
    required String title,
    required String body,
    String? payload,
    required Duration duration,
  }) async {
    await _plugin.zonedSchedule(
      id: id,
      title: title,
      body: body,
      scheduledDate: tz.TZDateTime.now(tz.local).add(duration),
      notificationDetails: _details,
      androidScheduleMode: await _scheduleMode(),
      payload: payload,
    );
  }

  static Future<void> scheduleDaily({
    required int id,
    required String title,
    required String body,
    required int hour,
    required int minute,
    String? payload,
  }) async {
    final now = tz.TZDateTime.now(tz.local);
```

```
    var scheduledDate = tz.TZDateTime(
      tz.local,
      now.year,
      now.month,
      now.day,
      hour,
      minute,
    );
    // Did the chosen time already pass today?
    bool timeAlreadyPassed = scheduledDate.isBefore(now);

    // If yes, move the reminder to tomorrow at the same time
    /*It's now 3:00 PM and the user picks 9:00 AM.
9:00 AM today has already passed, so timeAlreadyPassed = true.
The notification moves to 9:00 AM tomorrow. */
    if (timeAlreadyPassed) {
      scheduledDate = scheduledDate.add(const Duration(days: 1));
    }

    await _plugin.zonedSchedule(
      id: id,
      title: title,
      body: body,
      scheduledDate: scheduledDate,
      notificationDetails: _details,
      androidScheduleMode: await _scheduleMode(),
      payload: payload,

      matchDateTimeComponents:
          DateTimeComponents.time, // Daily at the same time
    );
  }

  // <<---------------------- Show Notification Periodically ---------->>
  static Future<void> showPeriodically({
    required int id,
    required String title,
    required String body,
    RepeatInterval interval = RepeatInterval.everyMinute,
    String? payload,
  }) async {
    await _plugin.periodicallyShow(
      id: id,
      title: title,
      body: body,
      repeatInterval: interval,
      notificationDetails: _details,
      androidScheduleMode: AndroidScheduleMode.inexactAllowWhileIdle,
      payload: payload,
    );
  }

  static Future<List<PendingNotificationRequest>> pending() =>
      _plugin.pendingNotificationRequests();

  static Future<void> cancel(int id) => _plugin.cancel(id: id);

  static Future<void> cancelAll() => _plugin.cancelAll();
```

How it works:

1. **Why timezone?** "8 PM" has to mean 8 PM in the **phone's** timezone. That's why `init()` calls `FlutterTimezone.getLocalTimezone()`.
2. `_scheduleMode()`: **exact vs. inexact**. Android may delay alarms to save battery. On Android 12+ the app asks for exact-alarm permission, which opens the "Alarms & reminders" settings page. If the user refuses, it falls back to inexact: the notification still arrives, maybe a little late.
3. `scheduleAfter()`: shows one notification after a duration, at `now + duration`.
4. `scheduleDaily()`:
   - If the chosen time already passed today, it moves to tomorrow.
   - `matchDateTimeComponents: DateTimeComponents.time` repeats it **every day** at that time.
5. `showPeriodically()`: fixed intervals only (every minute, hourly, daily, weekly). Our app doesn't use it, but it's good to know.
6. `pending()`: notifications that are scheduled but haven't shown yet.
7. `cancel(id)` cancels one notification. `cancelAll()` cancels everything.

3.2 `home_screen.dart`: "Remind me in 10 seconds"

Replace the empty `_remindIn10Seconds` with:

```dart
// DART

  // <<---------------------- Part 3: Quick scheduled test ---------->>
  Future<void> _remindIn10Seconds() async {
    await NotificationService.scheduleAfter(
      id: ReminderIds.quick,
      title: 'Study time 📚',
      body: 'This is a quick test reminder',
      duration: const Duration(seconds: 10),
      payload: 'quick_reminder',
    );
    if (mounted) _showMessage('Reminder in 10 seconds, close the app!');
  }
```

> **Try this:** tap it, **close the app**, and wait. The notification still arrives, because the **OS** shows it, not your app.

3.3 `home_screen.dart`: the daily reminder card

Replace the empty `_pickStudyTime` and `_cancelReminder` with:

```dart
// DART
  // <<---------------------- Part 3: Daily reminder ---------->>
  Future<void> _pickStudyTime() async {
    final time = await showTimePicker(
      context: context,
      initialTime: TimeOfDay.now(),
    );
    if (time == null) return; // user pressed cancel

    await NotificationService.scheduleDaily(
      id: ReminderIds.daily,
      title: 'Study time 📚',
      body: 'Open your notes, just 25 minutes!',
      hour: time.hour,
      minute: time.minute,
      payload: 'daily_reminder',
    );

    if (!mounted) return;
    setState(() => _reminderTime = time);
    _showMessage('Daily reminder at ${time.format(context)}');
  }
```

```dart
// DART
  // <<---------------------- Part 3: Cancel only the daily reminder ---------->>
  Future<void> _cancelReminder() async {
    await NotificationService.cancel(ReminderIds.daily);
    setState(() => _reminderTime = null);
    _showMessage('Reminder cancelled');
  }
```

How it works:

- `_reminderTime` lives in the state. While it's `null`, the card shows `--:--` and "No reminder set yet". After `setState`, it shows the big time with **Change time** and **Cancel**.
- `cancel(ReminderIds.daily)` cancels **only** the daily reminder, not the test ones. That's **why ids matter**.
- `mounted`: after an `await`, the user may have left the screen. Check `mounted` before using `context` or calling `setState`.

> **Try this:** pick a time 1 minute from now and wait for it. Then tap **Cancel**, and the reminder won't come again.

✓ **Checkpoint:** the core of the app works.

// part 04

Handling Notification Taps

Goal: tapping a reminder opens the Motivation screen.

4.1 The payload

You already attached payloads: `'test_notification'`, `'quick_reminder'`, and `'daily_reminder'`. Now you'll **read** them and show the value on the Motivation screen.

4.2 `notification_service.dart`: add `getLaunchPayload()`

Add this inside the class, at the end:

```dart
// DART
  static Future<String?> getLaunchPayload() async {
    final details = await _plugin.getNotificationAppLaunchDetails();
    if (details?.didNotificationLaunchApp ?? false) {
      return details!.notificationResponse?.payload;
    }
    return null;
  }
```

> If the app was **fully closed** and the user opened it by tapping a notification, this returns that notification's payload. Otherwise it returns `null`.

4.3 `lib/screens/motivation_screen.dart`

```dart
// DART
import 'dart:math';

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:local_notifications_app/core/app_theme.dart';

/// Opens when the user taps the study reminder notification.
class MotivationScreen extends StatelessWidget {
  const MotivationScreen({super.key, this.payload});

  final String? payload;

  static const _messages = [
    'Small steps every day lead to big results 💪',
    'Just 25 minutes, you can do it! ⏱️',
    'Your future self will thank you 🌟',
    'Consistency beats motivation 🔥',
  ];

  @override
```

```
  Widget build(BuildContext context) {
    final message = _messages[Random().nextInt(_messages.length)];

    // Light status bar icons on the purple background
    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: SystemUiOverlayStyle.light,
      child: Scaffold(
        body: Container(
          width: double.infinity,
          decoration: const BoxDecoration(gradient: AppColors.heroGradient),
          child: SafeArea(
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Column(
                children: [
                  Align(
                    alignment: Alignment.centerLeft,
                    child: IconButton(
                      onPressed: () => Navigator.maybePop(context),
                      icon: const Icon(Icons.close, color: Colors.white),
                    ),
                  ),
                  const Spacer(),
                  Container(
                    padding: const EdgeInsets.all(28),
                    decoration: BoxDecoration(
                      color: Colors.white.withValues(alpha: 0.15),
                      shape: BoxShape.circle,
                    ),
                    child: const Text('🏆', style: TextStyle(fontSize: 64)),
                  ),
                  const SizedBox(height: 32),
                  const Text(
                    'Time to study!',
                    style: TextStyle(color: Colors.white70, fontSize: 18),
                  ),
                  const SizedBox(height: 12),
                  Text(
                    message,
                    textAlign: TextAlign.center,
                    style: const TextStyle(
                      color: Colors.white,
                      fontSize: 28,
                      fontWeight: FontWeight.bold,
                      height: 1.3,
                    ),
                  ),
                  const SizedBox(height: 24),
                  // The payload we attached when we scheduled the notification
                  Container(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 14,
                      vertical: 6,
                    ),
                    decoration: BoxDecoration(
                      color: Colors.white.withValues(alpha: 0.15),
                      borderRadius: BorderRadius.circular(20),
                    ),
                    child: Text(
                      'payload: ${payload?.isNotEmpty == true ? payload : 'none'}',
```

```
                    style: const TextStyle(color: Colors.white70),
                  ),
                ),
                const Spacer(),
                SizedBox(
                  width: double.infinity,
                  child: FilledButton(
                    onPressed: () ⇒ Navigator.maybePop(context),
                    style: FilledButton.styleFrom(
                      backgroundColor: Colors.white,
                      foregroundColor: AppColors.primaryDark,
                      padding: const EdgeInsets.symmetric(vertical: 16),
                    ),
                    child: const Text(
                      "Let's study 📖",
                      style: TextStyle(
                        fontSize: 16,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
  }
}
```

4.4 lib/routing/routes.dart

```dart
// DART

/// Named-route constants. Use these instead of typing raw strings like
/// '/motivation'. The compiler catches a typo, a raw string won't.
class Routes {
  static const String home = '/home';
  static const String motivation = '/motivation';
}
```

4.5 `lib/routing/app_router.dart`

```dart
// DART
import 'package:flutter/material.dart';
import 'package:local_notifications_app/routing/routes.dart';
import 'package:local_notifications_app/screens/motivation_screen.dart';
import 'package:local_notifications_app/screens/home_screen.dart';

class AppRouter {
  // Nullable on purpose: returning null lets MaterialApp.onUnknownRoute
  // show the "page not found" screen.
  static Route<dynamic>? generateRoute(RouteSettings settings) {
    switch (settings.name) {
      // '/' can still arrive after a hot reload, so send it to Home too
      case '/':
      case Routes.home:
        return MaterialPageRoute(
          settings: settings,
          builder: (_) => const HomeScreen(),
        );

      case Routes.motivation:
        final payload = settings.arguments as String?;
        return MaterialPageRoute(
          settings: settings,
          builder: (_) => MotivationScreen(payload: payload),
        );

      default:
        return null;
    }
  }
}
```

4.6 `lib/main.dart` (final version)

Replace the whole file:

```dart
// DART
import 'package:flutter/material.dart';
import 'package:local_notifications_app/core/app_theme.dart';
import 'package:local_notifications_app/routing/app_router.dart';
import 'package:local_notifications_app/routing/routes.dart';
import 'package:local_notifications_app/services/notification_service.dart';

final navigatorKey = GlobalKey<NavigatorState>();

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await NotificationService.init();

  // App open (foreground/background) → user taps a notification
  NotificationService.onNotificationClick = (payload) {
    navigatorKey.currentState?.pushNamed(Routes.motivation, arguments: payload);
  };
```

```
  // App was closed → opened by tapping a notification
  final launchPayload = await NotificationService.getLaunchPayload();

  runApp(MyApp(launchPayload: launchPayload));
}

class MyApp extends StatelessWidget {
  const MyApp({super.key, this.launchPayload});

  final String? launchPayload;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Study Reminder',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      navigatorKey: navigatorKey,
      initialRoute: Routes.home,
      onGenerateRoute: AppRouter.generateRoute,
      onGenerateInitialRoutes: (initialRoute) => [
        AppRouter.generateRoute(RouteSettings(name: initialRoute))!,
        if (launchPayload != null)
          AppRouter.generateRoute(
            RouteSettings(name: Routes.motivation, arguments: launchPayload),
          )!,
      ],
      onUnknownRoute: (settings) => MaterialPageRoute(
        builder: (_) => Scaffold(
          appBar: AppBar(title: const Text('Page not found')),
          body: const Center(child: Text("We couldn't find that page.")),
        ),
      ),
    );
  }
}
```

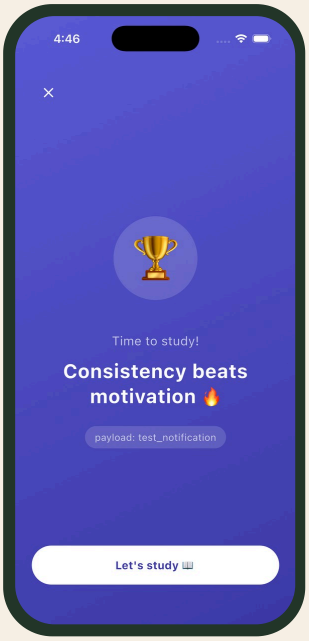
How it works:

- `navigatorKey`: the tap callback runs **outside** the widget tree, so it has no `context`. The key lets you navigate anyway.
- `onGenerateInitialRoutes`: if the app was opened from a notification, it starts with **Home + Motivation** on the stack, so the ✕ button still goes back to Home.
- **The 2 cases:**

APP STATE WHEN TAPPED	WHO HANDLES IT
Open or in the background	`onNotificationClick` (from `onDidReceiveNotificationResponse`)
Fully closed (killed)	`getLaunchPayload()` (from `getNotificationAppLaunchDetails`)

> **Try the full flow:** pick a study time → **swipe the app away** → the notification arrives → tap it → Motivation screen 🎉 → ✕ → Home.

✓ **Checkpoint:** your Study Reminder app is complete.



// opened from the notification

What's next: what if a server needs to send a reminder to every user? That's **push notifications with FCM**.

// ios

Running on iPhone

Good news: almost all of the code already works on iPhone. The Android steps (`build.gradle.kts` , the manifest permissions, and the receivers) are **Android only**, so you don't need them on iOS. On iPhone you only need the steps below.

1. `ios/Runner/AppDelegate.swift`

Full file. The important line is `UNUserNotificationCenter.current().delegate = ...`:

```swift
// SWIFT
import Flutter
import UIKit

@main
@objc class AppDelegate: FlutterAppDelegate, FlutterImplicitEngineDelegate {
  override func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
  ) -> Bool {
    UNUserNotificationCenter.current().delegate = self as UNUserNotificationCenterDelegate
    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
  }

  func didInitializeImplicitFlutterEngine(_ engineBridge: FlutterImplicitEngineBridge) {
    GeneratedPluginRegistrant.register(with: engineBridge.pluginRegistry)
  }
}
```

> This line connects iOS notification events to the plugin. Without it, notifications may not show while the app is open, and tapping a notification may not reach `onNotificationClick`.

2. Permission

Nothing new to add. `requestPermission()` already asks for alert, badge, and sound on iOS, and local notifications don't need any `Info.plist` entry.

- On iPhone, tap **Allow notifications** before testing. Until the user allows, nothing shows.
- iOS asks **only once**. If you tap "Don't Allow", turn it back on from **Settings → Notifications → Study Reminder**, or delete the app and install it again.

3. Showing notifications while the app is open

By default, iOS hides notifications while the app is open. These lines in `_details` (inside `notification_service.dart`) are what make them show:

```dart
// DART
    iOS: DarwinNotificationDetails(
      presentAlert: true,
      presentBanner: true,
      presentList: true,
      presentBadge: true,
      presentSound: true,
    ),
```

4. Run it

You need a **Mac with Xcode** to run on iOS.

iOS Simulator (local notifications work on the simulator):

```bash
// BASH
open -a Simulator
```

```bash
// BASH
flutter run
```

A real iPhone:

1. Open `ios/Runner.xcworkspace` in Xcode.
2. Select **Runner** → **Signing & Capabilities** → choose your **Team** (a free Apple ID works).
3. On the iPhone, turn on **Settings → Privacy & Security → Developer Mode** (iOS 16+), then connect it to the Mac.
4. Run `flutter run` and pick your iPhone.
5. If the iPhone says "Untrusted Developer", go to **Settings → General → VPN & Device Management** and trust your Apple ID.

Android vs iOS

TOPIC	ANDROID	IOS
Native setup	Desugaring + 3 manifest permissions + 2 receivers	One line in `AppDelegate.swift`
Permission	Asked on Android 13+	Always asked, and only once
Channels	Required ("Study Reminders")	No channels, the Android channel settings are ignored
On-time delivery	Needs exact-alarm permission	No extra permission
After the phone restarts	Needs `RECEIVE_BOOT_COMPLETED` + the boot receiver	iOS keeps scheduled notifications by itself
App is open	Notification shows	Shows because of `presentAlert` / `presentBanner`
Limit	No small limit	Max **64** pending scheduled notifications

// troubleshooting

Common Errors

SYMPTOM	CAUSE
Build fails with a desugaring error	Missing `isCoreLibraryDesugaringEnabled` / `coreLibraryDesugaring`
Instant notifications work, scheduled ones never show	Missing receivers in the manifest
Nothing shows at all	Permission denied, or the channel was turned off in settings
Scheduled notifications arrive late	No exact-alarm permission, so the app falls back to inexact mode
Changed the importance or sound and nothing changed	The channel was already created, so uninstall the app
`show(1, 'title', ...)` doesn't compile	Old tutorial API; v22 uses named parameters
Tapping from a killed app opens Home only	Missing `getLaunchPayload()` / `onGenerateInitialRoutes`
iOS: nothing shows at all	You didn't tap **Allow notifications**, or you denied it (turn it on in Settings)
iOS: nothing shows while the app is open	Missing the delegate line in `AppDelegate.swift`, or `presentAlert` / `presentBanner` is `false`
iOS: the permission dialog doesn't appear again	iOS asks only once. Use Settings, or delete and reinstall the app
Real iPhone: "Untrusted Developer"	Trust your Apple ID in **Settings → General → VPN & Device Management**

// practice

Extend Study Reminder

1. **Remember the time:** `_reminderTime` resets when the app restarts. Save it with `shared_preferences` and load it in `initState`.
2. **A reminder per subject:** Math, Physics, and so on, each with its own time, its own `id`, and its own Cancel button.
3. **Weekly reminder:** for example only on Fridays, using `DateTimeComponents.dayOfWeekAndTime`.
4. **Bonus:** pass the subject name in the `payload` and show it on the Motivation screen ("Time for Math! 📐").